fixtures/gate_shard/ — T (SpecKit- , plan.md T-C)

Contract: `specs/004-fast-dev-cycles/plan.md` §"T-C05 — Bounded parallel sharding" (lines 808-823). No standalone contract file exists for T-C05; plan.md's own T-C05 section IS the authoritative contract (matching the established project convention for tasks whose full spec lives in the plan rather than a separate `contracts/*.md` file).

Target tool (not yet implemented): `constitution/scripts/fastcycle/gates/gate_runner.sh` (shared with T-C04 "ordering" — T-C05 owns the `--mode shard` half). Neither `gates/gate_runner.sh` nor the `gates/` directory exists in this checkout yet (Setup phase T001-T006 has not landed) — this is CONFIRMED LIVE by Section A of `test_gate_shard_red.sh`, never assumed.

What plan.md T-C requires (verbatim, lines -)

> shards built from T-C02 write sets (no shard writes what another reads); `xargs -P N` with `N` computed at run time from `nproc`, §12.12 thread headroom and the §12.6 memory cap; shared resources (tracker DB, registry) are never written by gates in parallel.
>
> **Protecting tests:** sharded vs serial ×3 each → identical verdicts and identical canonical evidence hashes; golden-bad: a planted section pair sharing a temp file must be placed in one shard (or the test FAILs); paired mutation: ignore write sets → the golden-bad produces a flake and FAILs.

Fixture directories

Directory	Role	Proves
`gs_good_independent_determinism/`	golden / determinism	3 independent (disjoint-write-set) toy gates, once real, must produce the IDENTICAL verdict set + canonical evidence sha256 across 3 serial runs and 3 sharded runs (6 total)
`gs_bad_shared_temp_split/`	golden-bad	`gate_shared_writer_x.sh` + `gate_shared_writer_y.sh` both append to the SAME resolved shared temp path — MUST be co-scheduled into one shard by any correct sharder
`gs_negctrl_disjoint_temp/`	negative control	`gate_writer_p.sh` + `gate_writer_q.sh` write to two DIFFERENT resolved temp paths — MUST NOT be forced into the same shard (the §11.4.201(1) false-positive guard: an over-eager "always merge" sharder would incorrectly pass the golden-bad case above while defeating the parallelism T-C05 exists to deliver, and this fixture catches exactly that)

`_shared/gates/`

Toy gate scripts reused across fixture manifests (`manifest.json` 's `"script"` field names one of these, resolved relative to `_shared/gates/`):

- `gate_pass_a.sh` , `gate_pass_b.sh` — always exit 0, no I/O.
- `gate_fail_c.sh` — always exit 1 (so the determinism fixture's verdict set is a genuine PASS/FAIL mix, a stronger oracle than an all-PASS set).
- `gate_shared_writer_x.sh` , `gate_shared_writer_y.sh` — each appends a distinguishing line to `$1` (the manifest supplies the SAME resolved path to both, in `gs_bad_shared_temp_split/`).
- `gate_writer_p.sh` , `gate_writer_q.sh` — each appends a distinguishing line to `$1` (the manifest supplies DIFFERENT resolved paths to each, in `gs_negctrl_disjoint_temp/`).

Reference sharder — `../lib/shard_ref.py`

A from-scratch, independently-authored implementation of the write-set-intersection grouping rule plan.md line 812 states ("no shard writes what another reads") — union-find over declared `writes` paths, deterministic round-robin cluster-to-shard assignment. Producer != Verifier (§11.4.240): used ONLY to prove these three fixtures are non-vacuous BEFORE any claim is made about the (today, absent) real `gates/gate_runner.sh` 's behaviour. It never claims to BE the real sharding algorithm and never seeds or informs a future implementer's code — the reference was written and self-validated against hand-built manifests BEFORE this README or the main RED test file existed.

Self-validated 2026-09-28 (by hand, before being wired into the RED test): run against 3 independent gates at `n_shards=3` → each in its own shard; run against the shared-writer pair at `n_shards=3` AND `n_shards=1` → co-scheduled into one shard both times; run against the disjoint-writer pair at `n_shards=2` → correctly split into two DIFFERENT shards. Full transcript: this fork's own `qa-results/fastcycle/t055/red_run_evidence.log` .

Canonical evidence hash (used by `gs_good_independent_determinism/`)

`sha256(sorted("gate_name:VERDICT" for each gate, newline-joined))`, where `VERDICT` is `PASS` (exit 0) or `FAIL` (nonzero exit). This is DEFINED here (UNCONFIRMED by plan.md T-C05's own text — it names "canonical evidence hashes" but not the literal canonicalisation rule), binding-if- adopted, per the established house precedent (`fixtures/verdict_cache/` 's README.md documents the same kind of gap-fill for its own DEC-07 key schema and get/put wire format).

Paired mutation (§ . , once `gate_runner.sh` exists)

"Ignore write sets → the golden-bad produces a flake and FAILs" (plan.md line 820) — mutate the future `gate_runner.sh` to skip its write-set intersection check (always shard purely by gate-name hash, ignoring declared writes); re-run `gs_bad_shared_temp_split/` repeatedly — the mutated tool will, with nonzero probability, split `gate_shared_writer_x` and `gate_shared_writer_y` into different shards, producing a race on the shared temp path (a flaky, non-deterministic final file content), and the golden-bad assertion FAILs on that run. The unmutated tool must never flake on this fixture (deterministic co-scheduling every time, `n_shards` value irrelevant).

What this fixture set does NOT cover

- The REAL `xargs -P N` runtime-computed- `N` mechanism (nproc + §12.12 + §12.6) — that is an integration property of the real tool, asserted by `test_gate_shard_red.sh` Section A as a live control needle (confirming `xargs -P` itself works on this host today) but not independently re-implemented here.
- Tracker-DB / registry never-written-in-parallel — that invariant is already covered by the existing single-writer discipline (§11.4.206) and this feature's own T-B02/registry work; T-C05's fixtures assume it holds and do not re-test it.

- Load-balance QUALITY across shards (only CORRECTNESS of the co-scheduling constraint is tested here — plan.md T-C05 states no load-balance assertion).